

ft_irc 課題 学習・実装計画 v1

作成日: 2026-05-16
更新日: 2026-05-16
チーム: torinoue, sohyamaz, atashiro (3名)
担当: A=atashiro, B=torinoue, C1/C2=未定 (5/20決定予定)
成果物リポジトリ: myIRCd

読み分け: 本ファイルがメイン計画。進捗の数値・タスク表は本ファイル内で管理。セッションログは session_logs/ に記録。

基本方針

- **C++標準**: C++98 のみ (-std=c++98 でコンパイル可能であること)
- **I/O多重化**: poll() を使用 (macOS/Linux 両対応のため)
- **リファレンスクライアント**: irssi (後述の比較参照)
- **開発環境**: M2 Mac → 完成後に校舎環境へ移植
- **Mandatory パートのみ** (Bonus は対象外)

テストツール構成

結論: irssi + nc の併用

ツール	役割	用途
irssi	リファレンスクライアント	通常操作の動作確認、評価本番
nc (netcat)	テスト・デバッグ用	部分データ処理の検証、プロトコル手打ち

nc (netcat) について

課題書 IV.3 Test example で明示的に推奨されている：

```
$> nc -C 127.0.0.1 6667
com^Dman^Dd
$>
```

ctrl+D を使用してコマンドを複数の部分に分けて送信する：'com'、次に 'man'、次に 'd\n'。
コマンドを処理するには、まず受信したパケットを集約して再構築する必要がある。

nc の強み	説明
生のTCPソケット	IRCプロトコルを一切解釈しない
ctrl+D で部分送信	recv() が複数回に分かれる状況を再現
手動でコマンド入力	NICK alice\r\n を直接打てる
バグ発見に有効	バッファリング実装の不備を即座に露呈

評価者が nc テストを行う可能性が高い。部分データ処理の実装は必須。

irssi について

irssi の強み	説明
42学生の実績	GitHub上の ft_irc 参考実装の多くが irssi で動作確認
正しいIRCプロトコル	メッセージ形式が保証される
デバッグ容易	CLI なのでログが取りやすい
macOS対応	brew install irssi で即インストール

irssi 基本コマンド

```
# インストール
brew install irssi

# 接続
irssi -c 127.0.0.1 -p 6667 -w <password>

# サーバー内コマンド
/nick <nickname>
/join #channel
/msg <user> <message>
/part #channel
/quit
```

nc 基本コマンド

```
# 接続 (-C: CRLF を送信)
nc -C 127.0.0.1 6667

# 手動でIRCコマンドを入力
PASS mypassword
NICK alice
USER alice 0 * :Alice
JOIN #general
PRIVMSG #general :Hello!

# 部分データテスト (ctrl+D で分割送信)
PRI^DVMMSG #general :te^Dst
```

使い分け

場面	使うツール
開発中の動作確認	irssi
部分データ処理のテスト	nc
コマンド実装のデバッグ	nc (手打ちで確認)
評価本番	irssi + nc (評価者が nc テストを行う可能性あり)

GitHub 参考実装リスト

チームメンバー提供 (atashiro)

リソース	説明
mandakore/myIRCd	本プロジェクトの成果物リポジトリ。atashiro が作成・先行実装中
Qiita: 簡易サーバー例	C言語でのソケットサーバー実装例。基礎理解用

主要リポジトリ (概要のみ)

リポジトリ	特徴	参考度
barimehdi77/ft_irc	42学生、構造が明確、irssi対応	★★★
solMusic/ft_irc	42学生、ドキュメント充実	★★★
ft_irc-42	検索結果から複数参照可能	★★☆

注意: 参考実装は構造・設計パターンの参考に留める。コードのコピーは禁止。評価時に説明できないコードは使用しない。

参考にすべき設計パターン

- Client** クラス: fd, nickname, username, channels 等を管理
- Channel** クラス: name, topic, users, operators, modes を管理
- Server** クラス: clients, channels, poll fd 配列を管理
- Command** パーサー: IRC メッセージを解析してコマンドを実行

見積もり概要

BAC (Budget at Completion)

フェーズ	BAC	内容
0	8h	計画策定・環境整備
1	25h	ソケットプログラミング学習
2	15h	IRC プロトコル学習
3	25h	サーバー基盤実装
4	30h	コマンド実装
5	20h	テスト・デバッグ
6	5h	ドキュメント・README
7	10h	校舎確認・評価準備
合計	138h	

週間見積もり

週	時間	フェーズ	torinoue	sohyamaz
1	40h	0, 1	学習、基礎実装	学習、質問
2	40h	1, 2, 3	学習、サーバー基盤	学習、レビュー
3	40h	3, 4	コマンド実装	テスト、レビュー
4	18h	4, 5, 6, 7	仕上げ、デバッグ	テスト、評価準備

想定完了: 約3.5週間 (バッファ込みで4週間)

AI協働ルール

Inception と同様の Navigator/Driver モデルを採用。

用語	役割
Driver (ドライバー)	torinoue / sohyamaz (コードを書く人)
Navigator (ナビゲーター)	AI (方向を示す人)

実装タスクの進め方

1. **AIが解説・ヒントを出す** (概念、設計方針、確認すべきこと)
2. **AIが確認コマンドを提示** → ユーザーがターミナルで実行し結果を貼る
3. **AIが構造・ヒントを示す** → ユーザーがvim等でコーディング
4. **AIがレビュー** → 修正点を指摘
5. **ペア (torinoue/sohyamaz) で確認** → 両者が理解できること

フェーズ詳細

フェーズ 0: 計画策定・環境整備 (8h)

#	タスク	時間	担当	備考
0-1	課題書翻訳・理解	2h	両者	完了 (本セッション)
0-2	開発環境構築	2h	torinoue	irssi インストール、ディレクトリ構造
0-3	bircd 分析	2h	torinoue	課題添付サンプルの理解
0-4	参考実装調査	2h	両者	GitHub リポジトリの構造把握

フェーズ 1: ソケットプログラミング学習 (25h)

教材: 「TCP/IPソケットプログラミング C言語編」(オーム社)

- ・ [オーム社 書籍ページ](#) - 日本語版サンプルコード(CSockets.tar.gz)あり
- ・ [原著者サンプルコード](#) - 英語版全サンプル

li
| # | タスク | 時間 | 章/節 | 狙い | 備考 |
|--|-----|-----|-----|-----|
| 1-1 | 第1章: ネットワークとプロトコルの概要 | 1h | Ch.1 全体 | TCP/UDP/IP/ポートの基本概念 | 軽く読む |
| 1-2 | 第2章: ソケットの基礎 | 3h | Ch.2 全体 | socket/bind/listen/accept/send/recv | **重要:** TCPサーバー構造の核 |
| 1-3 | 第3章: メッセージの作成 | 1.5h | 3.1-3.2 | hton/ntohs、バイト順 | 3.3-3.4は軽く |
| 1-4 | 第5章: ノンブロッキングI/O | 2h | 5.3.1 | fcntl(O_NONBLOCK)、EAGAIN/EWOULDBLOCK | **重要:** 5.3.2(非同期I/O), 5.3.3(タイムアウト)は飛ばす |
| 1-5 | 第5章: 多重化 | 3h | 5.5 | select()の概念理解 → poll()への読み替え | **最重要:** ft_ircの核心 |
| 1-6 | 第6章: TCPの挙動 | 2h | 6.1, 6.4 | バッファリング、3-wayハンドシェイク、切断検知 | 6.2-6.3は参考程度 |
| 1-7 | 実践: poll()エコーサーバー | 4h | - | 学んだ内容を統合 | C++98、myIRCD/src/Server.cpp参照 |
| 1-8 | クイズ (ペア) | 1.5h | - | 相互確認 | 0100_socket_post_quiz.md |

スキップする章・節

- ・ 第4章: UDPソケット (全体) - ft_ircはTCPのみ
- ・ 5.3.2: 非同期I/O (SIGIO) - poll()を使う
- ・ 5.3.3: タイムアウト (setsockopt) - ノンブロッキング+poll()で対応
- ・ 5.4.1-5.4.2: fork/thread - ft_ircはfork禁止、poll()で多重化
- ・ 5.6: ブロードキャスト/マルチキャスト - IRC不要
- ・ 第7章: ドメインネームサービス - 必須ではない (余裕あれば)

書籍内容	読むべきコード	スキップ
Ch.2 TCP基礎	TCPEchoServer.c, HandleTCPClient.c	-
5.5 多重化	TCPEchoServer-Select.c (概念理解用)	-
-	-	UDPEcho*.c, -Fork.c, -Thread.c, -SIGIO.c, -Timeout.c, Broadcast.c, Multicast.c

フェーズ 2: IRC プロトコル学習 (15h)

RFC資料			
- RFC 1459 - Internet Relay Chat Protocol - IRCの基本仕様 - RFC 2812 - Internet Relay Chat: Client Protocol - クライアントプロトコル詳細			
#	タスク	時間	備考
2-1	RFC 1459 概要読解	3h	メッセージ形式、基本コマンド
2-2	RFC 2812 詳細読解	4h	クライアントプロトコル
2-3	irssi で実際のIRCサーバーに接続	2h	freenode 等で体験
2-4	Wireshark でプロトコル観察	3h	実際のパケットを確認
2-5	クイズ (ペア)	3h	相互確認

フェーズ 3: サーバー基盤実装 (25h)

#	タスク	時間	備考
3-1	Server クラス設計	3h	fd管理、poll配列
3-2	Client クラス設計	2h	接続情報、バッファ
3-3	Channel クラス設計	2h	メンバー、モード
3-4	socket/bind/listen/accept 実装	4h	接続受付
3-5	poll() ループ実装	5h	最重要: イベント駆動
3-6	部分データ処理	4h	バッファリング、CRLF検出
3-7	接続切断処理	3h	クリーンアップ
3-8	レビュー (ペア)	2h	コードウォークスルー

フェーズ 4: コマンド実装 (30h)

#	タスク	時間	備考
4-1	コマンドパーサー	4h	IRC メッセージ解析
4-2	PASS / CAP	3h	認証、capability negotiation
4-3	NICK / USER	4h	登録処理
4-4	JOIN / PART	4h	チャンネル参加/退出
4-5	PRIVMSG / NOTICE	4h	メッセージ送信
4-6	KICK	2h	オペレーター: 追放
4-7	INVITE	2h	オペレーター: 招待
4-8	TOPIC	2h	オペレーター: トピック
4-9	MODE (+i, +t, +k, +o, +l)	5h	オペレーター: モード

フェーズ 5: テスト・デバッグ (20h)

#	タスク	時間	備考
5-1	nc テスト (部分データ)	3h	課題書のテスト例
5-2	irssi 接続テスト	4h	リファレンスクライアント
5-3	複数クライアント同時接続	4h	ハング確認
5-4	エッジケーステスト	5h	不正入力、切断、リソースリーク
5-5	メモリリークチェック	2h	valgrind / leaks
5-6	ペアテスト	2h	sohyamaz による破壊テスト

フェーズ 6: ドキュメント・README (5h)

#	タスク	時間	備考
6-1	README.md 作成	3h	課題書要件に準拠
6-2	AI使用の記載	1h	Resources セクション
6-3	最終コードレビュー	1h	コメント整理

フェーズ 7: 校舎確認・評価準備 (10h)

#	タスク	時間	備考
7-1	校舎マシンでビルド確認	3h	環境差異の対応
7-2	評価シミュレーション	4h	想定質問への回答準備
7-3	ディフェンス修正練習	3h	即座にコード修正できるか

クイズ計画

quizzes/ ディレクトリに以下を作成予定：

ファイル名	対象フェーズ	内容
0100_socket_pre_quiz.md	1	ソケット基礎（書籍読了前）
0100_socket_post_quiz.md	1	ソケット基礎（書籍読了後）
0200_irc_protocol_pre_quiz.md	2	IRC プロトコル（RFC 読了前）
0200_irc_protocol_post_quiz.md	2	IRC プロトコル（RFC 読了後）
0300_server_impl_quiz.md	3	サーバー実装確認
0400_command_impl_quiz.md	4	コマンド実装確認

セッションログ命名規則

session_logs/NNNN_session_log_ft_irc.md

- NNNN：0パディング4桁の連番（0000, 0001, 0002...）
- 毎セッション終了時に作成

セッションログ必須記載事項

項目	説明
開始時刻 / 終了時刻	JST で記録
実稼働時間	1h 単位
参加者	torinoue / sohyamaz / AI のうち誰が参加したか
完了タスク	タスク番号と内容
AI議事録	不在者への共有用（参加者が1名の場合）

新しいチャット開始時のコピペ用指示文

ft_irc課題（42Tokyo）を進めています。
チーム：torinoue, sohyamaz

以下を読んで現在地を把握してから作業を始めてください：

- dev_docs/phase_plan.md（全体計画）
- session_logs/ 内の最新セッションログ

今日やること：[タスク番号とタスク名をここに書く]

未解決事項

事項	確認方法	タイミング	状態
評価シートの入手	Web検索 or 先輩に確認	フェーズ 5 開始前	解決: 42evalhub から入手済。 docs/eval/evalsheet_42evalhub.md 参照
校舎マシンの C++ バージョン	校舎で確認	フェーズ 7	未着手
CAP コマンドの要否	irssi 接続時に確認	フェーズ 4-2	未着手
リファレンスIRCクライアント選定	チームで協議	フェーズ 2 開始前	未着手（irssi仮決定、要確認）